

Turbo Charge Python Apps with Speed



Part-2

In the first part of this series, we tried out some of the tools to turbo-charge mainly CPU-bound Python programs, without modifying the source code too often. Another way to boost run-time performance is to leverage the run-time parallelism provided by modern multi-core machines with SMP-enabled operating systems. This article is to get you started with parallel computing in Python on multi-core/multi-processor systems and computing clusters.

If you can design your Python application so that independent parts of it run in parallel on multi-core machines, then it will boost application performance a lot. If your existing Python applications are multi-threaded, or you write your new code to exploit concurrency, then it could have run in parallel on multi-core machines. Unfortunately, the current CPython implementation can't provide run-time parallelism for Python threads on multi-core machines. This limitation is due to the Global Interpreter Lock (GIL); to understand this, please refer to Part I of this series, which appeared in the July 2010 issue of *LFY*. However, I'll present ways to conquer the GIL issue, so you can get started with parallel computing.

First, you need some understanding of parallel computing. A good starting point is the Wikipedia link mentioned in the *References* section. I used the Ubuntu 9.10 x86_64 desktop distribution, with Python 2.7, to test the code in this article and to generate the screen-shots.

Overcoming the GIL to extract performance

The GIL limits the number of the threads that can execute at a time in CPython, to one, impairing the ability to run in parallel on multi-core machines. The way to overcome this limitation is to use sub-processes in place of threads, in your Python code. Although this sub-process model does have creation and inter-process communication

(IPC) overheads, it frees you from the GIL issue, and is adequate most of the time.

If this seems complicated, don't worry; most of the time, you won't have to deal with the arcane lower-level details of sub-process creation, and the IPC between them. All the alternatives we discuss here are internally based upon this multi-process model; the only difference between them is the level of abstraction provided to parallelise your Python code.

Multi-threading in the multi-processing way

The easiest way to turn Python code that's designed with threading, into an application that will use multi-core systems, is to use the inbuilt *multiprocessing* module, which is included in Python version 2.6 onwards. This module's API is similar to the *threading* module, so you need to make very few changes, due to the similarity. Though there's some lower-level work involved, there is no need to install any additional Python components.

You have to use the *multiprocessing* module's *Process* class to create a new sub-process object. The *start* method of this object executes a function that's passed to it in the sub-process, with the tuple of arguments that's passed. There is also a *join* method, to wait for the completion of your sub-process. Save the code shown below into *basic.py*, and run it (*python basic.py*, or make the script executable with *chmod u+x basic.py* and run *./basic.py*). You can clearly